PROJECT TECHNICAL DOCUMENTATION

BioSpecInfo

Web platform for computational chemistry, physics and mathematics

Deploy Guide

DOCUMENT CONTROL

Identifier	`BSI-DOC-12-EN`
Title	Deploy Guide
Software version	**bsi-v248**
Revision (commit)	`cd8df42`
Date of issue	2026-11-11
Author	Samuele Pio Provenzano
Language	English
Status	Issued with version bsi-v248

Document generated from the versioned source in the project repository. The version and revision shown here and in the footer of every page identify exactly the state of the software described.

Contents

0. What gets published, in practice

1. Environments

2. Publishing to production

- 2.1 Procedure
- 2.1-bis Publishing the version's release
- 2.1-ter Why the version bump is not done with a find-and-replace
- 2.2 The two version lines
- 2.2-bis Large files outside the precache
- 2.2-ter One secret phrase, two fingerprints
- 2.2-quater A new module: three places to keep aligned
- 2.2-quinquies Spectra Studio, a page of its own
- 2.3 How the update reaches users

3. Publishing alternatives

- A rule that holds everywhere: sw.js must not be cached

4. Optional proxy — a separate deployment

- Recommended configuration

5. Domain and certificate

6. Monitoring

7. Rollback and emergency procedures

- 7.1 Going back to the previous version
- 7.2 "I published but I do not see the changes"
- 7.3 "The assistant has stopped answering"
- 7.4 Restoring the user's data

8. Pre-publication checklist

Deploy Guide — BioSpecInfo

Field	Value
Software	BioSpecInfo
Version described	`bsi-v248`
Purpose	Operational procedure for publishing, updating and rolling back the application.

0. What gets published, in practice

BioSpecInfo is a set of **static files**. There is nothing to compile, no process to keep running, no database to provision.

Build	None. The repository files *are* the application.
Server runtime	None
Environment variables	None for the application. Only the optional proxy has any (§4).
Database	None server-side
Server requirement	Serve static files over HTTPS with the correct MIME types

The one non-obvious requirement: the server must return `application/wasm` for `.wasm` files, otherwise RDKit and SQLite do not start. GitHub Pages, Netlify, Vercel and Cloudflare Pages already do.

1. Environments

Environment	How it is obtained	What it is for
Development	`python3 -m http.server 8899` from the root	Daily work and verification benches
Preview	A branch published on Pages, or `npx serve`	Checking before merging
Production	GitHub Pages from the `main` branch	`samupropio1-ship-it.github.io/BioSpecInfo-v11/`

> **Why a server is needed even in development.** Opening `index.html` with a double click (`file://`) does not work: RDKit and SQLite are WebAssembly modules and require an HTTP context. It is the reason the verification benches also run against a local server.

2. Publishing to production

2.1 Procedure

```
# 1. from the working branch, the full battery
python3 -m http.server 8899 &
node tools/genera-evidenza.js          # must close with 0 failures

# 2. bump the version — with the tool, NEVER by hand
node tools/porta-versione.js bsi-v<NNN+1> --prova   # look first
node tools/porta-versione.js bsi-v<NNN+1>           # then write

# 3. regenerate SBOM, evidence, PDFs and packages — IN THIS ORDER
#    The packages bundle the PDFs and the evidence: built first, they carry
#    the typeset documents and the report of the PREVIOUS version.
#    genera-pacchetti now notices and stops, but better not to get there.
node tools/genera-sbom.js
node tools/genera-evidenza.js       # the battery; needs a server on :8899
node tools/genera-pdf.js            # 36 PDFs; every footer carries version and commit
node tools/genera-pacchetti.js      # the three packages; fails on broken links, stale PDFs
or stale evidence

# 4. merge into main
git add -A && git commit -m "…"
git push -u origin <branch>
#    then pull request and squash merge
```

GitHub Pages publishes automatically from `main`: the *pages build and deployment* workflow takes about a minute.

2.1-bis Publishing the version's release

The application is online as soon as `main` updates; the **release** serves a different purpose: giving an assessor a fixed point to download — a tag, the version notes and the delivery packages attached.

It is published from the repository, not from outside:

> **Actions → release → Run workflow**, field `versione`: `bsi-v<NNN>`.

The workflow creates the tag on the current commit, opens the release and attaches the three packages from `consegna/`. The full archive — `models/` included, about 124 MB — is generated by GitHub itself as *Source code (zip)*.

Why from the repository and not from the command line. The credentials a remote session uses on the repository are authorised to write to branches and **not to tags**: the `POST` to `git-receive-pack` for `refs/tags/…` answers `403 Forbidden`, while a new branch goes through. The Actions token, by contrast, writes to the repository that hosts it.

What the workflow **refuses** to do:

Condition	Outcome
`CHANGELOG.md` has no `## [bsi-v<NNN>]` entry with content	fails: a release without notes looks like a version and does not say what changed
fewer than three packages in `consegna/`	fails: a missing attachment would make the release silently incomplete
the tag does not point at the commit the packages came from	fails: a release must have a single provenance

That last row exists because of a real defect. On re-running the workflow to refresh the packages, `gh release edit` updated notes and assets but **did not move the tag**: the *Source code (zip)* archive, which GitHub generates from the tag, stayed at the first publication's commit while the packages came from a later one. Two provenances, and nothing saying so. The tag is now moved, and a following step does not take that on trust: it asks GitHub where the tag points and compares.

The workflow is **re-runnable**: if the release already exists it updates notes, assets and tag instead of failing halfway. Moving an already-published tag is not free — whoever downloaded it already has different content — so it applies to a release just published, not an old one: for that, publish a new version.

2.1-ter Why the version bump is not done with a find-and-replace

It looks like the most harmless operation in the world: search `bsi-v<NNN>`, write `bsi-v<NNN+1>`. **For three releases in a row it was done that way, and every time it rewrote historical sentences** — sentences recording a fact that happened at a past version. What came out of it sat inside a signed conformance statement:

the document said	the truth
the password had been **changed at the current version**	it was changed at version `bsi-v188`
the tag had been **moved to the current version's commit**	the incident happened at version `bsi-v188`
the thirty-six entries were **added in the current version**	they were added in `bsi-v188`
"bump `CACHE` (e.g. `bsi-vN` → `bsi-vN`)"	an example equal to itself

No check could catch it: every line named a version that **exists**, in the document describing that version.

> This table uses placeholders rather than the real numbers because the two checks described below **would reject it** — which is the proof that they work.

`tools/porta-versione.js` touches **only** the lines that DECLARE the version — headers, footers, the README badge, package names, the two code lines — and **lists** everything else so it gets read. With `--prova` it writes nothing.

Two checks in `tools/verifica-documenti.js` now close the door: no past-tense sentence may name the current version, and no upgrade example may show the same version on both sides.

> **Name the thing, not the number.** "From the Pretsch tables" rather than "from version such-and-such": it reads better and does not age.

2.2 The two version lines

`CACHE` in `sw.js` and `BSI_APP_VERSION` in `index.html` **must match**.

- `CACHE` determines whether the Service Worker considers the cache stale:

without a bump, devices keep serving the old files and the publication reaches nobody.

- `BSI_APP_VERSION` is what the user sees in the "Updates" window. «Check for

updates» compares the active Service Worker's version (it answers the `BSI_VERSIONE` message with its `CACHE`) with the `var CACHE = '…'` line of the published `sw.js`, read from the same site with no cache and, only if that does not answer, from GitHub. `BSI_APP_VERSION` enters the comparison only when no Service Worker answers.

> **The shape of the line matters.** The check reads `sw.js` with the expression `var CACHE = '…'` . Up to version `bsi-v236` it looked for a `CACHE_VERSION` line that `sw.js` no longer contained, and the published version was never read. If the declaration in `sw.js` changes shape, `bsi-aggiorna.js` must change too; `test_controllaagg` notices.

They drifted apart once in the past — the second had been stuck for months — and the result was that the update check was comparing the wrong number.

2.2-bis Large files outside the precache

`models/spectra-noir.glb` (Spectra's face, about 8 MB) is not in the Service Worker's precache, like the other `models/*.glb`: it is downloaded the first time the app derives Spectra's icon or opens the Spectra Live console. It must be published with the rest, with its CC BY 4.0 attribution (`07-SBOM.md`); if it is missing, the console uses the fallback face.

2.2-ter One secret phrase, two fingerprints

Since version `bsi-v238` the same secret phrase serves the File Manager and the recognition of Spectra's creator. Its SHA-256 fingerprint is written in two places: `file_manager.html` and `bsi-spectra-creatore.js` (constant `IMPRONTA`). If the phrase changes, **both** must change in the same release; otherwise the File Manager opens with the new phrase and Spectra recognises only the old one. A recognition mark already written in a browser (`bsi-spectra-creatore-v1`) is not revoked by the change: it is removed with «forget that I am your creator» or with «Clear data». The plaintext phrase must never be written to a file in the repository.

2.2-quater A new module: three places to keep aligned

A new application `.js` file, such as `bsi-lezione.js` (the guided lesson), must be declared in `index.html` (with `defer`; `bsi-lezione.js` comes before `bsi-spectra-live.js`, which uses its `BSILezione` object when it receives «fammi lezione»), in the `PRECACHE` list of `sw.js`, so that it works offline, and in the module list of `tools/genera-sbom.js`, so that it appears in the SBOM. A module missing from the precache works while there is a network and disappears offline.

The latest module added this way is `bsi-spectra-esplora.js` (v242, «⚡ Explore with Spectra»): in `index.html` with `defer`, **after** `bsi-fisica-comune.js`, because it registers its three sections (`sesplchim`, `sesplfis`, `sesplmat`) with `BSIFisica.sezione` as soon as it runs; in the `PRECACHE` of `sw.js`, next to `bsi-spectra-spiega.js`; in the list of `tools/genera-sbom.js`. The three containers `<div id="sesplchim" class="section">`… and the three bar buttons are in `index.html`. The module brings no 3D files of its own: it uses the shared figure of `bsi-spectra-figura.js`.

In the same version `bsi-spectra-olointerattivo.js` (v242, the «⛶ Interagisci» interactive hologram) follows the same three rules: in `index.html` with `defer`, **right after** `bsi-spectra-figura.js`, which creates the button on the hologram and moves the figure into its window; in the `PRECACHE` of `sw.js`; in the list of `tools/genera-sbom.js`. It uses three.js and the figure already loaded by `bsi-spectra-figura.js` and the local assistant (`bsi-assistente.js`, `bsi-mat-cas.js`), and brings no data files of its own: molecules and planets are in the code.

2.2-quinquies Spectra Studio, a page of its own

`spectra-studio.html` (v239) does not go through `index.html`: it loads by itself `three.min.js`, `gltf_loader.js`, `bsi-spectra-mente.js`, `bsi-spectra-creatore.js`, `bsi-spectra-androide.js`, `bsi-spectra-corpo.js`, `bsi-spectra-rig.js`, `bsi-spectra-figura.js`, `bsi-spectra-voce.js` and `bsi-spectra-studio-lezioni.js`. The page and these modules are in the `PRECACHE` of `sw.js`; `bsi-spectra-figura.js` (v240) is also in `index.html`, with `defer`, because it shows Spectra with the body in the AI sections: it loads `three.min.js`, `gltf_loader.js` and the other 3D modules only the first time one of those sections is opened; `bsi-spectra-spiega.js` (v240, the «🧠 Spiega» tab of the Spectra window) is also in `index.html` with `defer` and in the precache; `bsi-spectra-voce.js` is also in `index.html`, because the console and the guided lesson use it. The studio's «Noir» face is the same `models/spectra-noir.glb` as the console's, outside the precache (§2.2-bis): if it does not arrive, the body stays without a face.

Spectra's body (v241): `models/spectra-corpo.bsi` must be deployed. The «Bounty Hunter» body (free on CGTrader, Royalty Free licence: used inside the product, not offered as a standalone file) is embedded in the app as the packed block `models/spectra-corpo.bsi` (4.40 MB), which is in the repository and must be deployed with the app: without it the figure and the studio stay on «Spectra non è arrivato: serve la rete la prima volta» with «↻ Riprova». The block is not a `.glb`: only `bsi-spectra-figura.js` (function `apri`) reopens it. Like the `models/*.glb` files it is not in the precache: the service worker (network first) caches it on the first request, and from then on it works offline. The original `.glb` (`models/spectra-corpo.glb`) stays out of git (`.gitignore`); the studio tries it only in development, if the block is missing. The `models/spectra-cyberpunk.glb` fallback no longer exists.

Regenerating the block (for example after changing the model):

```
node tools/impacchetta-corpo.js path/to/body.glb models/spectra-corpo.bsi
# ✓ models/spectra-corpo.bsi (4.40 MB), verificato: si riapre identico al .glb
```

The tool checks that the input is a `.glb`, writes the block («BSIC» · version · length · bytes mixed with an xorshift32 stream from the fixed seed, the same as in `bsi-spectra-figura.js`) and checks that it reopens identical. Then the new `models/spectra-corpo.bsi` is deployed; clients' caches refresh with the `sw.js` version change. A change of model must also be reflected in `models/CREDITS.md` and `THIRD_PARTY_NOTICES.md`. The quality voice calls `api.openai.com` directly from the browser: any `Content-Security-Policy` (`03-Security-Privacy-Compliance.md` §2.4) must allow it in `connect-src`, otherwise the voice falls back on the browser's synthesis.

2.3 How the update reaches users

1. The browser detects a different `sw.js` and installs the new Service Worker.
2. `skipWaiting()` activates it at once; the old caches are deleted.
3. The Service Worker **notifies** the open tabs (`BSI_SW_UPDATED`) — it does not

reload them on its own authority.

1. The page decides: if no work is in progress it reloads itself; if the

assistant is answering or there is unsaved text, it shows "Update now / Later".

Whoever opens the app at that moment already receives the new version.

---

3. Publishing alternatives

The application does not depend on GitHub Pages. Any static file host will do, provided it is over HTTPS (the Service Worker requires it).

Platform	Command / configuration	Notes
GitHub Pages	Settings → Pages → branch `main`, folder `/`	In use. Free, custom domain possible
Netlify	`netlify deploy --prod --dir=.`	No build command
Vercel	`vercel --prod`	Static project, no framework
Cloudflare Pages	`npx wrangler pages deploy .`	Convenient if the proxy is already in use
VPS with nginx	Copy the files to `/var/www/biospecinfo`	See the configuration below

nginx configuration

```
server {
    listen 443 ssl http2;
    server_name biospecinfo.example.org;
    root /var/www/biospecinfo;

    # Without this MIME type, RDKit and SQLite do not start
    types { application/wasm wasm; }

    # sw.js must not be cached by the CDN: it is what announces new versions
    location = /sw.js {
        add_header Cache-Control "no-cache, no-store, must-revalidate";
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

A rule that holds everywhere: `sw.js` must not be cached

If a CDN keeps `sw.js`, the browser keeps seeing the old version and **no update ever arrives**, however much one publishes. This holds on every platform.

---

4. Optional proxy — a separate deployment

The proxy is a **distinct** component, owned by the user, published on their own Cloudflare account. It is not required in order to use the application.

```
cd proxy
npx wrangler login
npx wrangler deploy
# the address is printed: https://spectra-proxy.<account>.workers.dev

npx wrangler secret put GROQ_KEYS      # one or more keys, comma-separated
npx wrangler secret put GEMINI_KEYS    # optional
```

The address is then pasted into the assistant's "Proxy" box, inside the app. The same four steps are available **inside the application**, with the commands ready to copy.

Recommended configuration

```
# Without this variable the proxy accepts requests from any origin
npx wrangler secret put ORIGINI
# value: https://samupropio1-ship-it.github.io
```

Variable	Default	Why change it
`ORIGINI`	*(empty = all)*	**Always set it.** An open proxy is usable by anyone, at the expense of your quota.

Variable	Default	Why change it
`LIMITE_IP`	`20`	Requests per IP within the window
`TETTO_GIORNO`	`2000`	Overall daily ceiling

> **Keys must never go into `wrangler.toml`.** That file is versioned: a key written there ends up on GitHub, and the bots that scan it find it within hours. Use `wrangler secret put`, which keeps them out of the repository.

---

5. Domain and certificate

Aspect	Situation
HTTPS	Mandatory: without it the Service Worker does not register and the app does not work offline
Certificate	Provided and renewed automatically by GitHub Pages
Custom domain	Settings → Pages → Custom domain, plus a `CNAME` record pointing to `<user>.github.io`

---

6. Monitoring

There is no server to monitor: there are no processes that could fall over, nor resources that could run out.

What to watch	How
Outcome of the publication	GitHub → Actions → *pages build and deployment*
Availability	An HTTP check on `index.html` (UptimeRobot or similar)
Client-side errors	The browser console. Remote telemetry exists but is **disabled** (`BSI_TELEMETRY_URL` empty): enabling it requires a deliberate decision about data handling.
Proxy consumption	Cloudflare → Workers → Analytics

---

7. Rollback and emergency procedures

7.1 Going back to the previous version

```
git revert <commit>
git push origin main
```

Pages republishes in about a minute. Devices receive the restored version as a normal update — **provided `CACHE` changes**: a revert that also takes the version number backwards is fine, because what counts is that the value be *different*, not that it be greater.

7.2 "I published but I do not see the changes"

In order:

1. **Was the version bumped?** It is the cause in ninety per cent of cases.

Without it, the Service Worker serves the cache.

1. **Has Pages finished?** GitHub → Actions.
2. **Is `sw.js` cached by a CDN?** It must answer `no-cache`.
3. From the app: ✨ menu → 🔄 **Updates** → **Check for updates**. It tells

which version is served and which is published; if they differ it forces a `registration.update()`. If it answers «up to date» with the old number, the `sw.js` the site serves is still the old one (points 2 and 3).

1. As a last resort: developer tools → Application → Service Workers →

Unregister, then reload.

7.3 "The assistant has stopped answering"

This is not a publishing problem: it concerns the AI provider. In the app, the 🔌 **Test** button measures in a few seconds which providers the device can actually reach, without needing any key.

7.4 Restoring the user's data

The data live on the device and **there are no copies on a server**: they cannot be restored remotely. This is a direct consequence of the local-first architecture, and it is declared to the user. Exports (notes, documents) remain the only form of backup.

---

8. Pre-publication checklist

- ☐ `node tools/genera-evidenza.js` closes with **0 failures**
- ☐ `CACHE` in `sw.js` and `BSI_APP_VERSION` in `index.html` bumped and **equal**
- ☐ `node tools/genera-sbom.js` re-run
- ☐ `node tools/genera-pdf.js` re-run — otherwise the attachments describe the previous version
- ☐ `node tools/genera-pacchetti.js` re-run and closed with **0 broken links**
- ☐ Evidence report regenerated against the commit being published
- ☐ Any new deviations noted in `09-Release-Conformance-Statement.md` §4
- ☐ No conflict markers (`verifica_guida` checks this across 58 files)
- ☐ No API key in the versioned files

- ☐ The accessibility debt **has not grown** — if it has fallen, regenerate the

baseline with `node tools/verifica-accessibilita.js --aggiorna-riferimento` and record it in the commit

- ☐ Code coverage **has not fallen** — `audit_copertura` fails if it has

Document updated to version `bsi-v248`.